

OS PRACTICAL 3

Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

```
shravya@Shravya: ~  
shravya@Shravya:~$ nano fork3.c  
shravya@Shravya:~$ gcc fork3.c  
shravya@Shravya:~$ ./a.out  
Parent Process  
Parent Process ID is 723  
Parent's Parent Process ID is 325  
Child Process  
Child Process ID is 724  
Parent Process ID is 323  
shravya@Shravya:~$
```

During the execution of the program, the `ps` command was used to view the processes currently running in the terminal. It displayed the process IDs and the corresponding commands.

```
shravya@Shravya: ~  
shravya@Shravya:~$ nano fork3.c  
shravya@Shravya:~$ gcc fork3.c  
shravya@Shravya:~$ ./a.out  
Parent Process  
Parent Process ID is 723  
Parent's Parent Process ID is 325  
Child Process  
Child Process ID is 724  
Parent Process ID is 323  
shravya@Shravya:~$ ps  
  PID TTY          TIME CMD  
  325 pts/0        00:00:00 bash  
  566 pts/0        00:05:35 a.out  
  665 pts/0        00:01:01 a.out  
  724 pts/0        00:00:17 a.out  
  734 pts/0        00:00:00 ps  
shravya@Shravya:~$
```

Used the top command to continuously monitor the processes while they were executing. It provided a real-time view of the processes and their activity. This helped in observing how the process status changes during execution.

```

734 pts/0    00:00:00 ps
shravya@Shravya:~$ top
top - 16:14:33 up 9 min,  1 user,  load average: 2.59, 1.26, 0.54
Tasks:  28 total,   4 running, 24 sleeping,   0 stopped,   0 zombie
%Cpu(s): 25.0 us,  0.0 sy,  0.0 ni, 74.8 id,  0.0 wa,  0.0 hi,  0.2 si,  0.0 st
MiB Mem : 15836.6 total, 14506.7 free,   598.7 used,   929.4 buff/cache
MiB Swap: 4096.0 total,  4096.0 free,    0.0 used. 15237.9 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
566	shravya	20	0	2768	1064	968	R	100.0	0.0	6:17.76	a.out
665	shravya	20	0	2768	1068	968	R	100.0	0.0	1:44.49	a.out
724	shravya	20	0	2768	1068	968	R	100.0	0.0	1:00.03	a.out
1	root	20	0	24144	15288	11492	S	0.0	0.1	0:00.58	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.00	init-systemd(Ub
6	root	20	0	3180	2044	1940	S	0.0	0.0	0:00.00	init
46	root	19	-1	50428	16748	15488	S	0.0	0.1	0:00.14	systemd-journal
78	systemd+	20	0	22540	14608	11988	S	0.0	0.1	0:00.06	systemd-resolve
85	root	20	0	35460	12388	9188	S	0.0	0.1	0:00.38	systemd-udev
104	root	20	0	2888	1948	1820	S	0.0	0.0	0:00.05	chronyd-starter
105	root	20	0	4472	3148	2900	S	0.0	0.0	0:00.00	cron
106	message+	20	0	8820	5388	4716	S	0.0	0.0	0:00.05	dbus-daemon
110	root	20	0	44092	29752	14608	S	0.0	0.2	0:00.25	networkd-dispat
114	root	20	0	18432	9424	8176	S	0.0	0.1	0:00.09	systemd-logind
121	root	20	0	1792300	15172	12552	S	0.0	0.1	0:00.10	wsl-pro-service
187	syslog	20	0	220548	5916	4632	S	0.0	0.0	0:00.07	rsyslogd
194	root	20	0	5492	2724	2536	S	0.0	0.0	0:00.00	agetty
210	root	20	0	123456	32600	17992	S	0.0	0.2	0:00.19	unattended-upgr
217	_chrony	20	0	20508	7856	7016	S	0.0	0.0	0:00.02	chronyd
227	_chrony	20	0	12092	2444	1800	S	0.0	0.0	0:00.00	chronyd
322	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00	SessionLeader
323	root	20	0	3200	1248	1108	S	0.0	0.0	0:00.15	Relay(325)
325	shravya	20	0	6268	5524	3848	S	0.0	0.0	0:00.10	bash
326	root	20	0	8648	5552	4708	S	0.0	0.0	0:00.01	login
369	shravya	20	0	22332	13592	10976	S	0.0	0.1	0:00.10	systemd
371	shravya	20	0	22912	4072	2068	S	0.0	0.0	0:00.00	(sd-pam)
401	shravya	20	0	6136	5488	3928	S	0.0	0.0	0:00.02	bash
752	shravya	20	0	8188	6020	3900	R	0.0	0.0	0:00.00	top

Used the /proc filesystem to inspect the process with PID 1, which was the fork1 process identified using the ps command. The /proc/1 directory contains different files and directories that provide information about the process, such as its status, memory, scheduling, file descriptors and other process-related details. This helped me observe how Linux maintains information about an active process.

```
shravya@Shravya: ~  
shravya@Shravya:~$ ls /proc/1  
arch_status  exe      maps     pagemap   stack  
attr         fd       mem      patch_state  stat  
auxv        fdinfo  mountinfo personality  statm  
cgroup      gid_map mounts   projid_map  status  
clear_refs  io      mountstats  root        syscall  
cmdline     ksm_merging_pages net      sched       task  
comm        ksm_stat ns       schedstat   timens_offsets  
coredump_filter latency numa_maps sessionid   timers  
cpuset      limits  oom_adj   setgroups  timerslack_ns  
cwd         loginuid oom_score smaps      uid_map  
environ     map_files oom_score_adj smaps_rollup wchan  
shravya@Shravya:~$
```